

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – INCIDENT ANALYSIS (ROOT CAUSE & RISK)

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO5 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Incident Investigation, Root Cause Analysis and Risk Assessment.		
Student Name:	Harish T	Reg. No.:	192521373
Section / Batch:	B Tech IT 2025	Date:	27/08/26

Instructions to Students

- Review the given security incident scenario, evidence, logs and indicators carefully before beginning the analysis.
- Identify the incident sequence, affected assets, relevant indicators and supporting evidence without making unsupported assumptions.
- Determine the root cause by linking observed events, vulnerabilities and contributing conditions in a logical sequence.
- Assess identified risks based on likelihood, severity and potential technical or operational impact.
- Present investigation findings, root-cause reasoning and risk assessment clearly with appropriate technical justification.

Question Type	Focus Area	Questions	Marks
INCIDENT ANALYSIS	Investigation & Risk	Q1	Q1 –100
GRANDTOTAL:		100 Marks	

Code of Conduct

I Harish T (192521373)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Harish T

ASSESSMENT TOOL 2 – INCIDENT ANALYSIS

(ROOT CAUSE & RISK)

CSA5803 – DevSecOps Engineering and Application Security

Analyze an Airline Reservation System experiencing automated booking requests and service degradation. Develop an incident response plan, recommend mitigation strategies, and justify bot protection, rate limiting, autoscaling, and DDoS controls.

1. Incident Overview and Nature of the Problem

The airline reservation system is experiencing automated booking requests and a corresponding degradation in service quality. The incident should be treated as an availability and application-security event because the abnormal request activity is affecting the ability of the reservation platform to serve legitimate customers. The first responsibility of the incident team is not simply to block traffic, but to determine what is happening, which services are affected, why performance is deteriorating, and how the organization can restore normal operation without interrupting genuine passengers.

The investigation must remain evidence based. The available scenario establishes automated booking requests and service degradation, but it does not by itself prove the exact attack mechanism, source, volume, or whether the event is a distributed denial-of-service attack. Therefore, those details should be confirmed through application logs, network telemetry, WAF/CDN records, infrastructure metrics and database monitoring. This distinction is important because an effective incident report separates confirmed observations from hypotheses and contributing factors.

2. Affected Assets

- The airline reservation web application and customer-facing booking interfaces.
- Public and internal APIs used for flight search, availability, booking creation and related reservation functions.
- API gateways, load balancers, WAF/CDN services and other traffic-management components.
- Application servers, containers, virtual machines or cloud instances running reservation services.
- Databases, caches, connection pools and other backend dependencies supporting searches and bookings.
- Authentication, session-management and payment-related services if excessive traffic reaches these components.
- Monitoring, logging and security infrastructure required to detect and investigate the incident.

3. Business and Technical Impact

The immediate technical impact is increased workload and degraded service performance. Excessive requests can consume CPU, memory, worker threads, connection pools and database capacity. As queues increase, response time can rise and timeouts or server errors may occur. If the reservation service is overloaded, legitimate customers may be unable to search flights, view seats, create reservations or complete a booking.

The business impact can be significant because an airline reservation platform is directly connected to revenue generation. Failed or abandoned bookings may result in lost sales, increased customer complaints and reduced confidence in the airline's digital services. A prolonged incident can also increase operational costs when emergency infrastructure scaling, additional support activity or specialist incident-response resources are required.

4. Incident Sequence

- Automated clients generate repeated or unusually frequent requests to reservation-related endpoints.
- The application accepts and processes those requests, consuming compute, network, connection and database resources.
- Resource utilization and request queues increase, causing latency and error rates to rise.
- Legitimate customers experience slow pages, timeouts, failed searches or unsuccessful booking operations.
- Monitoring and security systems detect abnormal traffic and service-health changes.
- The incident-response team correlates traffic and infrastructure evidence, identifies the dominant request patterns and determines the appropriate containment level.
- Bot protection, rate limiting, WAF/CDN controls and other mitigations are applied while autoscaling supports the remaining legitimate workload.
- Service health is verified, normal thresholds are gradually restored and a post-incident review identifies long-term improvements.

5. Investigation Strategy and Evidence Collection

A reliable investigation should correlate multiple evidence sources rather than relying on one log or one metric. The response team should establish the exact time at which degradation began and compare that period with a normal baseline. This allows the team to determine whether request volume, request type or resource consumption changed significantly.

- Web and API access logs: request counts, URLs/endpoints, HTTP methods, status codes, response time and request frequency.
- WAF and CDN telemetry: challenges, blocked requests, bot scores, rule matches and edge request volume.
- Load-balancer data: incoming request rate, backend health, connection counts and traffic distribution.
- Application metrics: CPU, memory, worker utilization, queue length, thread pools, connection pools and error rates.
- Database metrics: connection usage, query rate, slow queries, lock contention, CPU and cache effectiveness.
- Cloud/platform metrics: instance count, autoscaling events, network utilization and resource saturation.
- Authentication/session logs: whether requests originate from valid users, repeated sessions, suspicious account activity or anonymous automation.
- Security monitoring: alerts, source patterns, user-agent characteristics and previously observed abusive behavior.

Useful indicators include a sharp increase in requests per second, repeated access to a small number of expensive endpoints, unusually high request rates from individual identities or sources, highly repetitive request patterns, abnormal session behavior, increased 4xx/5xx responses and resource saturation. These indicators should be time-correlated before the team concludes that the event is malicious.

6. Root Cause Analysis

From the scenario alone, the most defensible immediate-cause hypothesis is that excessive automated booking requests are generating an abnormal workload that the reservation platform is struggling to process. The complete root cause should be confirmed from evidence. The causal chain can be represented as: automated requests → increased API/application workload → resource pressure and queueing → increased latency/errors → service degradation for legitimate users.

The investigation should also identify contributing conditions. If evidence shows that the system had no effective bot protection, weak rate limits, expensive unauthenticated operations, insufficient caching or inadequate capacity controls, these should be recorded as contributing factors rather than incorrectly described as the initiating event. Similarly, if the traffic is found to be distributed across many sources or to saturate network capacity, DDoS-related

factors may be confirmed. The final root-cause statement should therefore distinguish observed facts, technical causes and control weaknesses.

7. Risk Assessment

The principal risk is loss of availability. The risk becomes more serious when expensive reservation operations are accessible at high frequency because a relatively small amount of traffic may create disproportionate backend work. A second risk is cascading failure: overloaded application services may increase database connections and retries, which can place additional pressure on downstream components.

A practical risk ranking is:

- Critical – automated request overload: high likelihood in the stated scenario and high availability impact.
- Critical – application/API resource exhaustion: high likelihood when excessive traffic is processed and high impact on service continuity.
- High – database overload: medium to high likelihood if requests trigger expensive queries, with high impact because the database can become a shared bottleneck.
- Critical – legitimate customer disruption: high likelihood once service degradation is occurring, with high business impact.
- High – infrastructure cost increase: autoscaling can increase capacity and operating cost during sustained abnormal traffic.
- High – secondary operational/security impact: degraded systems can make monitoring, support and other dependent services harder to operate.

8. Incident Response Plan – Preparation

Preparation should be completed before an incident. The airline should maintain centralized logging, application and infrastructure dashboards, WAF/CDN policies, bot-management capability, rate-limit configurations, autoscaling policies and DDoS-provider integration. Responsibilities should be clearly assigned to the security, application, infrastructure, database and business teams. The organization should also maintain a tested incident runbook describing who can activate emergency controls and who approves changes.

9. Identification and Triage

When alerts indicate service degradation, the incident team should first confirm the affected services and establish a timeline. Request rate, latency, error rate, CPU, memory, database utilization and connection usage should be checked together. The team should determine whether the traffic is concentrated on booking endpoints, whether it comes from a limited or distributed set of sources, and whether the requests display automation characteristics.

Triage should also consider legitimate explanations such as a major fare release, promotional campaign, holiday travel peak or another unusual demand event. This prevents the team from blocking genuine customers merely because traffic is high. The key distinction is not simply high volume, but whether the traffic pattern is consistent with legitimate use and whether the request behavior is causing abnormal resource consumption.

10. Containment

- Apply targeted rate limits to high-risk and resource-intensive endpoints.
- Enable or tighten bot-management and WAF rules for clearly automated or abusive traffic.
- Challenge suspicious clients rather than immediately blocking broad network ranges when legitimate users may share addresses.

- Restrict expensive operations through authentication, quotas, concurrency limits and request validation where appropriate.
- Protect the database by limiting excessive concurrent requests and preventing unnecessary retries.
- Prioritize critical booking transactions so non-essential traffic cannot consume disproportionate capacity.

Containment should be gradual and measurable. Every restrictive control should be monitored for false positives and customer impact. The objective is to reduce abusive load while preserving legitimate booking functionality.

11. Eradication, Recovery and Lessons Learned

After containment, the team should correct the weaknesses that allowed the abnormal traffic to consume excessive resources. This can include tuning bot rules, redesigning rate limits, optimizing expensive queries, introducing caching, improving authentication controls and correcting capacity policies. Recovery should be gradual rather than an immediate removal of all controls.

During recovery, monitor booking success rate, response time, HTTP errors, CPU, memory, database load, request volume and blocked/challenged traffic. Once the service remains stable, normal thresholds can be restored carefully. A post-incident review should document detection time, containment time, customer impact, false positives, infrastructure cost and control effectiveness, then convert those findings into updated runbooks and security controls.

12. Mitigation Strategy – Bot Protection

Bot protection is the most direct control for the stated automated-booking problem. A bot-management system can evaluate behavioral and request-level signals to identify clients that behave differently from normal passengers. Depending on confidence, suspicious traffic can be allowed, challenged, throttled or blocked. This reduces the amount of unwanted traffic that reaches expensive reservation functions.

The main advantage is precision. A simple IP block can accidentally affect legitimate passengers because mobile networks, corporate networks and NAT environments can cause many users to appear behind the same address. Bot protection can use multiple signals and behavioral patterns instead of relying only on a single IP address. It is therefore useful for distinguishing normal browsing from repeated automated workflows.

Bot protection should be stronger around sensitive and expensive operations such as repeated availability queries, booking creation and automated reservation attempts. The rules should be tested carefully because an overly aggressive challenge can harm genuine customers, especially users with accessibility requirements or unusual but legitimate browsing behavior.

13. Mitigation Strategy – Rate Limiting

Rate limiting controls how frequently a client or identity can send requests during a defined period. It is a preventive control that limits the amount of work any one source can impose on the reservation system. Limits can be applied by IP, account, session, API key, token or another suitable identity, depending on the architecture.

Rate limits should be endpoint-specific. A flight-search operation may legitimately occur more frequently than booking creation, payment initiation or account-sensitive operations. For example, an application may permit a higher search rate while imposing a much stricter limit on repeated booking attempts. When a client exceeds its limit, the API can return HTTP 429 and record the event for monitoring.

Rate limiting should ideally occur at the edge or API gateway so excessive requests are rejected before they consume application and database resources. It should also be adjustable because legitimate demand can vary significantly. Combining rate limiting with bot detection is stronger than using either control independently: bot detection identifies suspicious behavior, while rate limiting limits the damage that remains.

14. Mitigation Strategy – Autoscaling

Autoscaling dynamically adds or removes application capacity according to measured demand. This is valuable for an airline because legitimate booking demand can change quickly. During a genuine traffic spike, additional application instances can distribute the workload, reduce queueing and help maintain availability.

However, autoscaling must not be considered a security control by itself. If automated traffic continues to rise, autoscaling can respond by creating additional instances, increasing infrastructure cost without removing the underlying abuse. Downstream components such as databases may also become bottlenecks even when application capacity increases. Therefore, autoscaling should operate together with bot protection, rate limiting, caching, concurrency controls and database safeguards.

Autoscaling policies should include sensible minimum and maximum capacity, appropriate scale-out triggers and scale-in protection so that capacity does not fluctuate unnecessarily. The organization should also monitor cost and downstream resource limits during an incident.

15. Mitigation Strategy – DDoS Controls

DDoS controls protect the reservation environment from large volumes of unwanted traffic that can overwhelm network, transport or application resources. Network and upstream DDoS protection is important when traffic volume is large enough to threaten bandwidth or infrastructure. CDN and WAF services can filter traffic at the edge before it reaches the origin application.

Application-layer abuse requires additional controls because an attacker may send requests that appear technically valid but are designed to consume expensive application resources. For this reason, DDoS controls should be layered with bot management, rate limiting and API protections rather than treated as a replacement for them.

The airline should maintain an escalation procedure for activating stronger DDoS mitigation when network or application traffic crosses predefined thresholds. The procedure should include monitoring, communication, technical ownership and recovery steps. DDoS protection should also be tested periodically so that the team understands how traffic is filtered and how legitimate users are protected.

16. Why the Four Controls Must Work Together

Control	Main function	Justification in this incident
Bot protection	Identify and control automated clients	Directly addresses the automated booking requests and reduces abusive traffic before expensive processing.
Rate limiting	Control request frequency	Stops individual clients, accounts or identities from consuming unlimited API/application capacity.
Autoscaling	Increase application capacity	Provides resilience when demand is legitimate or when controlled residual traffic temporarily increases.
DDoS controls	Protect network and edge infrastructure	Reduces the impact of large-scale traffic and prevents excessive traffic from reaching the origin.

These controls address different layers of the problem. DDoS protection works at the network and edge, bot protection identifies automation, rate limiting constrains request frequency, and autoscaling supplies additional capacity. Because each control has a different purpose, the strongest design is a defense-in-depth model rather than dependence on one technology.

17. Recommended Layered Architecture

A suitable request flow is: Internet → DDoS protection → CDN/WAF → bot-management controls → API gateway/rate limiter → load balancer → autoscaled reservation services → cache/database protection. The principle is to reject or control unwanted traffic as early as possible. Requests that are allowed to proceed should encounter additional safeguards before reaching expensive backend components.

Caching should be used where the data and consistency requirements permit it, particularly for repeated read-heavy searches. Database protection should include connection limits, query optimization and controls against excessive concurrency. The architecture should also provide centralized monitoring so the incident team can see how much traffic is blocked at each layer and how much reaches the application.

18. Final Recommendation

The recommended response is an evidence-driven, layered incident-response strategy. The team should first establish the actual traffic pattern and affected resources through logs and monitoring. Immediate containment should use targeted rate limits, bot/WAF controls and protection for expensive endpoints. DDoS mitigation should be activated when traffic characteristics indicate network or application-layer flooding. Autoscaling should maintain availability for legitimate demand, but it should have sensible limits and downstream safeguards so that abusive traffic cannot simply create unlimited infrastructure consumption.

The effectiveness of the response should be measured using concrete outcomes: restoration of normal response time, reduced error rates, stable CPU and database utilization, improved legitimate booking success, controlled request rates, acceptable false-positive levels and predictable infrastructure cost. The post-incident review should then update the organization's security policies, incident runbooks, alert thresholds, rate limits, bot rules, scaling policies and DDoS procedures.

19. Conclusion

An airline reservation system exposed to automated booking requests requires both incident response and preventive engineering. The incident should be investigated through correlated application, network and infrastructure evidence so that confirmed facts are separated from assumptions. The most likely immediate problem is excessive automated workload causing resource pressure and service degradation, but the final root cause should be confirmed through evidence.

Bot protection is justified because it identifies and controls automation. Rate limiting is justified because it prevents excessive request frequency. Autoscaling is justified because it provides resilience during legitimate demand spikes, although it must not be used as the sole response to abuse. DDoS controls are justified because they protect the network, edge and origin from large-scale traffic. Together, these controls create layered protection that improves availability, reduces abuse impact and supports reliable airline booking services.